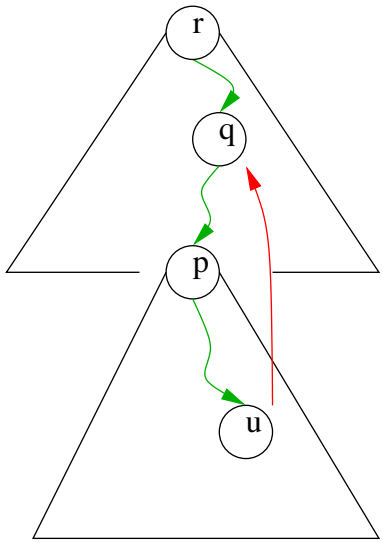
Gráfalgoritmusok 3
Elvágópontok, hidak, kétszeresen összefüggő komponensek

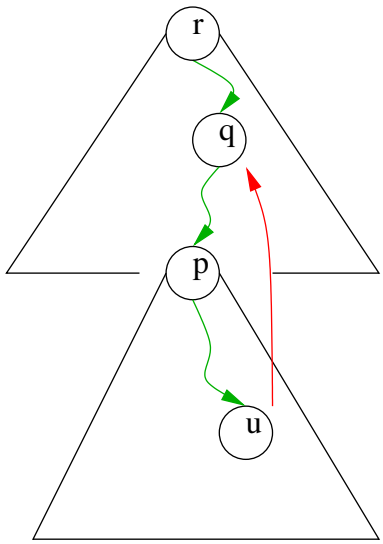
Szerző: Horváth Gyula
horvath@inf.elte.hu



1. ábra

1.0.1. L függvény

Legyen `G = (V, E)` irányítatlan gráf, és tekintsük egy `r`-gyökerű `MFF` mélységi feszítőfáját. Legyen `D(p)` a `p` elérési ideje a mélységi bejárás során. Definiáljuk az $\mathtt{L} : \mathtt{V} \to \mathbb{N}$ függvényt a következőképpen:



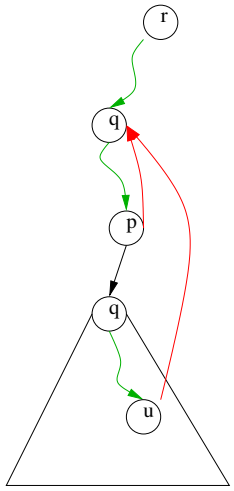
2. ábra

$$\mathtt{L(p)} = \min \begin{cases} \mathtt{D(p)} \\ \mathtt{D(q)} \ : \ \text{van olyan } \mathtt{u} \text{ leszármazottja } \mathtt{p}\text{-nek, hogy } \mathtt{u} \to \mathtt{q} \text{ visszaél} \end{cases}$$

Állítás

Az L függvény kiszámítható az élek számával arányos időben (O(|E|)) a mélységi bejárás során.

$$L(p) = \min_{p \to q} \begin{cases} D(p) & \\ D(q) : & p \to q \text{ visszaél} \\ L(q) : & p \to q \text{ faél} \end{cases}$$



3. ábra

1.1. Az L függvény kiszámítása

```
1  void MelyBejar(int p){
2  //Globális: G, ido, D,Szin,L
3    D[p]=++ido;
4    L[p]=ido;
5    Szin[p]=Szurke;
6    for (int q:G[p])
7      if (Szin[q]==Feher){//p->q faél
8        FeFa[q]=p;
9        MelyBejar(q);
10       L[p]=min(L[p],L[q]);
11   }else{
12     if(Szin[q]==Szurke && FeFa[p]!=q && D[q]<L[p])//visszaél
13       L[p]=D[q];
14   }
15   Szin[p]=Fekete;
16 }
```

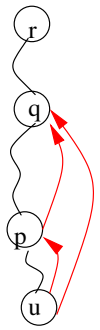
2. Összefüggőségi kérdések irányítatlan gráfokban

2.1. Körben lévő pontok

Feladat: határozzuk meg a `G = (V, E)` irányítatlan gráf azon pontjait, amelyek benne vannak legalább egy körben!

2.1. Állítás. *Irányítatlan gráf bármely mélységi bejárása esetén minden él vagy faél, vagy visszaél.*

Tekintsünk egy tetszőleges, `r` gyökerő mélységi bejárást. Ha a bejárás során a `p` $\rightarrow$ `q` él vizsgálatánál `q` színe szürke, akkor minden olyan `v` pont, amely a feszítőfában `p` és `q` között van (beleértve `p`-t és `q`-t) körben van. Fordítva is igaz, ha `v` benne van legalább egy körben, akkor a mélységi bejárás során előáll a fenti eset.



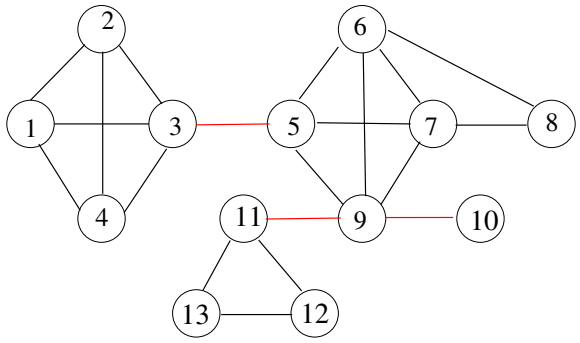
4. ábra

2.2. Állítás. *Egy* `G = (V, E)` *irányítatlan gráf* `p` $\in$ `V` *pontja akkor és csak akkor van körben, ha a mélységi bejárás során van* `p` $\rightarrow$ `q` *visszaél, vagy van olyan* `u` *leszármazottja, hogy abból van olyan* `u` $\rightarrow$ `q` *visszaél, hogy* `q` *az* `r` *és* `p` *között van a feszítőfában.*

2.3. Állítás. *Irányítatlan gráf bármely* `p` *pontja akkor és csak akkor van benne legalább egy körben, ha* `L(p)` $<$ `D(p))` *vagy* `p`*-nek van olyan* `u` *fia a mélységi feszítőfában, hogy* `L(u)` $\leq$ `D(p)`.

2.2. Elvágópontok

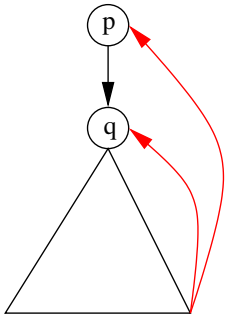
2.4. definíció. *Egy* `G = (V, E)` *összefüggő irányítatlan gráf* `p` $\in$ `V` *pontját elvágópontnak nevezzük, ha* `p`*-t (és a hozzá kapcsolódó minden élet) törölve, a gráf nem lesz összefüggő.*



5. ábra. Elvágópontok: `3, 5, 9, 11`

2.5. Állítás. *Irányítatlan gráf bármely* `p` *pontja akkor és csak akkor elvágópont, ha* `p` $\neq$ `r` *és van olyan* `q` *fia a mélységi feszítőfában, hogy* `L(q)` $\geq$ `D(p)`.

2.6. Állítás. *Irányítatlan gráf esetén a mélységi feszítőfa gyökere akkor és csak akkor elvágópont, ha egynél több fia van a feszítőfában.*



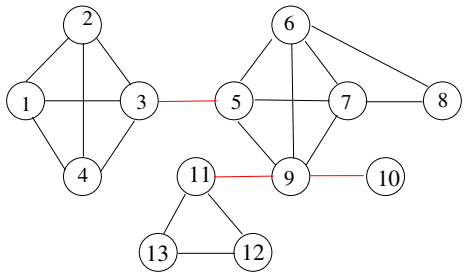
6. ábra. p elvágópont esetei

2.2.1. Elvágópontok kiszámítása

```
#include <bits/stdc++.h>
#define maxN 100001
using namespace std;
vector<int> G[maxN];
int D[maxN];
int L[maxN];
int ido=0;
enum Paletta {Feher, Szurke, Fekete};
Paletta Szin[maxN];
bool ElvagoPontok[maxN];
```

```
1  void MelyBejar(int p, int os){
2  //Global: G, ido, D, Szin, L, ElvagoPontok
3    D[p]=++ido;
4    L[p]=ido;
5    Szin[p]=Szurke;
6    for(int q:G[p]){
7      if(q==os) continue;
8      if(Szin[q]==Feher){//p->q faél;
9        MelyBejar(q, p);
10       L[p]=min(L[p],L[q]);
11       if(L[q]>=D[p]){//
12         ElvagoPontok[p]=D[p]>1 || D[q]>2;
13       }
14     }else if(Szin[q]==Szurke){//p->q visszaél
15       L[p]=min(L[p],D[q]);
16     }
17   }
18   Szin[p]=Fekete;
19 }
20 int main(){
21   int n,m,x,y;
22   cin>>n>>m;
23   for(int i=0;i<m;i++){
24     cin>>x>>y;
25     G[x].push_back(y);
26     G[y].push_back(x);
27   }
28   for(int x=1;x<=n;x++) Szin[x]=Feher;
29   MelyBejar(1,0);
30
31   for(int x=1;x<=n;x++) if(ElvagoPontok[x]) cout<<x<<",";
32   cout<<endl;
33 }
```

2.3. Hidak



7. ábra. Hidak: `3 — 5, 11 — 9, 9 — 10`

2.7. definíció. *Egy irányítatlan* `G = (V, E)` *gráf* `(p, q)` $\in$ `E` *éle akkor és csak akkor híd, ha az élet elhagyva a gráf nem lesz összefüggő.*

2.8. Állítás. *Egy irányítatlan* `G = (V, E)` *gráf* `(p, q)` $\in$ `E` *éle akkor és csak akkor híd, ha nincs benne egyetlen körben sem.*

2.9. Állítás. *Egy irányítatlan* `G = (V, E)` *gráf* `(p, q)` $\in$ `E` *éle akkor és csak akkor híd, ha* `L(q) = D(q)`.

2.10. következmény. *Egy gráf minden hídja az élek számával arányos időben kiszámítható.*

```
//Irányítatlan gráf hidjai
#include <bits/stdc++.h>
#define maxN 100001
using namespace std;
vector<int> G[maxN];
int D[maxN];
int L[maxN];
int ido=0;
enum Paletta {Feher, Szurke, Fekete};
Paletta Szin[maxN];
vector<pair<int,int>> Hidak;
```

```
34 void MelyBejar(int p, int os){
35   //Global: G, ido, D,Szin,L, Hidak
36   D[p]=++ido;
37   L[p]=ido;
38   Szin[p]=Szurke;
39   for (int q:G[p]){
40     if(q==os) continue;
41     if (Szin[q]==Feher){//p->q fa él
42       MelyBejar(q, p);
43       L[p]=min(L[p],L[q]);
44       if (L[q]==D[q])
45       Hidak.push_back({p,q});
46     }else if(D[q]<L[p])//visszaél
47     L[p]=D[q];
48   }
49 }
50 //
51 int main (){
52   int n,m,x,y;
53   cin>>n>>m;
54   for(int i=0;i<m;i++){
55     cin>>x>>y;
56     G[x].push_back(y);
57     G[y].push_back(x);
58   }
59   for(int x=1;x<=n;x++) Szin[x]=Feher;
60   MelyBejar(1,0);
61   cout<<Hidak.size()<<endl;
62   for(auto e: Hidak)
63   cout<<e.first<<" "<<e.second<<endl;
64 }
```

3. Összefüggőség irányítatlan gráfban

Ebben a fejezetben, ha másképp nem jelöljük, akkor gráfon mindig irányítatlan gráfot értünk. Azt mondjuk, hogy egy `G = (V, E)` gráf összefüggő, ha bármely két pontja között van út.

3.1. Pont-összefüggőség

3.1. definíció. *Azt mondjuk, hogy a* $G = (V, E)$ *gráf 2-pont-összefüggő, ha összefüggő és bármely pontját törölve a gráf továbbra is összefüggő marad. Másképpen fogalmazva, a gráfban nincs elvágópont.*
A kétszeresen összefüggő (biconnected) és a 2-összefüggő elnevezés is használatos.

3.2. definíció. *Kétszeresen összefüggő komponens olyan maximális részgráf, amely kétszeresen összefüggő.*
A 2-összefügő komponenseket blokkoknak is nevezik.

A kétszeresen összefüggő komponensek tulajdonságai

Minden $G = (V, E)$ kétszeresen összefüggő gráfra teljesül az alábbi hat állítás.

3.3. Állítás. *Bármely két* $v_1, v_2 \in V$ *ponthoz van olyan kör, amely mindkét pontot tartalmazza.*

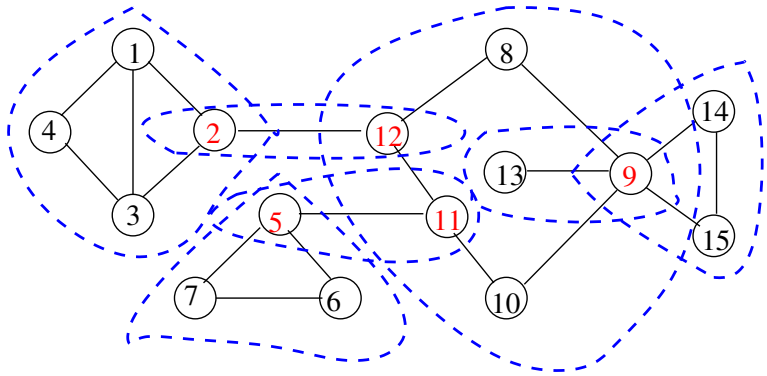
3.4. Állítás. *Bármely két* $e_1, e_2 \in E$ *élhez van olyan kör, amely mindkét élet tartalmazza.*

3.5. Állítás. *Bármely* $(v_1, v_2) \in E$ *él és* $v \in V$*ponthoz van olyan kör, amely a* (v_1, v_2) *élet és a* v *pontot is tartalmazza.*

3.6. Állítás. *Bármely három* $v_1, v_2, v_3 \in V$ *ponthoz van olyan út* v_1*-ből* v_2*-be, amely* ***tartalmazz*** v_3*-at;* $v_1 \rightsquigarrow v_3 \rightsquigarrow v_2$

3.7. Állítás. *Bármely három* $v_1, v_2, v_3 \in V$ *ponthoz van olyan út* v_1*-ből* v_2*-be, amely* ***nem tartalmazza*** v_3*-at;* $v_1 \rightsquigarrow v_2$

3.8. Állítás. *Bármely két* $v_1, v_2 \in V$ *pont és* $e \in E$ *él esetén van olyan út* v_1*-ből* v_2*-be, amely* ***tartalmazza*** *az* e *élet.*



8. ábra. Kétszeresen összefüggő komponensek

3.2. A 2-összefüggő komponensek meghatározása

Az algoritmus elve hasonló az erősen összefüggő komponensek meghatározásánál használt Tarjan algoritmushoz. Amikor a mélységi bejárás során a $p \to q$ faélet vizsgáljuk és azt detektáljuk, hogy a p pont elvágópont, akkor a veremből kivesszük a pontokat q-ig, ezek a pontok képeznek egy komponenst (blokkot).

3.2.1. Megvalósítás

```
1  //Irányítatlan gráf 2-összefüggő (biconnected) komponensek
2  #include <bits/stdc++.h>
3  #define maxN 10001
4  using namespace std;
5
6  int n;//pontok száma
7  vector<int> G[maxN];
8  void BeOlvas();
9
10 enum Paletta {Feher, Szurke, Fekete};
11 Paletta Szin[maxN];
12 int D[maxN];
13 int L[maxN];
14 stack<int> V;
15 vector<vector<int>> Blokk;
16 int ido;
```

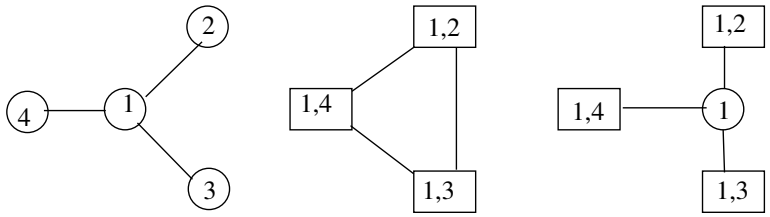
```
65 void MelyBejar(int p, int os){
66 //Global: G, ido, D,Szin,L, V,Blokk
67   D[p]=++ido;
68   L[p]=ido;
69   Szin[p]=Szurke;
70   V.push(p);
71   for (int q:G[p]){
72     if(q==os) continue;
73     if (Szin[q]==Feher){//p->q faél;
74       MelyBejar(q, p);
75       L[p]=min(L[p],L[q]);
76       if (L[q]>=D[p]){//találtunk egy komponenst
77         Blokk.push_back({p});
78         while (Blokk.back().back() != q) {
79           Blokk.back().push_back(V.top());
80           V.pop();
81         }
82       }
83     }else if (q!=os && Szin[q]==Szurke){//p->q visszaél
84       L[p]=min(L[p],D[q]);
85     }
86   }
87   Szin[p]=Fekete;
88 }
89
90 void Komponensek(){
91   //Globális: Szin
92   for (int p=1; p<=n; p++) {Szin[p]=Feher;}
93   ido=0;
94   for (int p=1; p<=n; p++){
95     if (Szin[p]==Feher) MelyBejar(p,-1);
96   }
97 }
```

```
int main (){
  ios_base::sync_with_stdio(false);cin.tie(NULL);
  BeOlvas();
  Komponensek();
  cout<<Blokk.size()<<endl;
  for(auto b:Blokk){
    for(int p:b) cout<<p<<" "; cout<<endl;
  }
}

void BeOlvas(){
  //Global:n,G
  int m,p,q;
  cin>>n>>m;
  for (int i=0; i<m; i++){
    cin>>p>>q;
    G[p].push_back(q);
    G[q].push_back(p);
  }
}
```

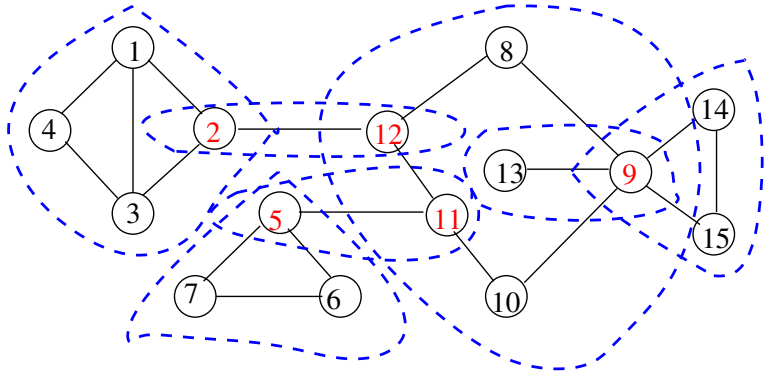
3.3. Blokk-elvágó fa

Látható, hogy a kétszeresen összefüggő komponensek nem feltétlenül diszjunktak. Pontosabban, bármely két komponensnek legfeljebb egy közös pontja lehet, ami elvágópont. Mivel az elvágópontok kapcsolják össze a komponenseket, ezért hasonlóan az irányított gráfok esetéhez, itt is beszélhetünk komponensek gráfjáról. Ez az a gráf, amelynek pontjai a kétszeresen összefüggő komponensek, az élek pedig a közös elvágópontok. Ez a gráf azonban nem olyan kedvező tulajdonságú, nem fa. Ezt a gráfot blokk-gráfnak is hívják.
Azonban, ha külön pontnak tekintjük az elvágópontokat, akkor már ez a gráf fa lesz.

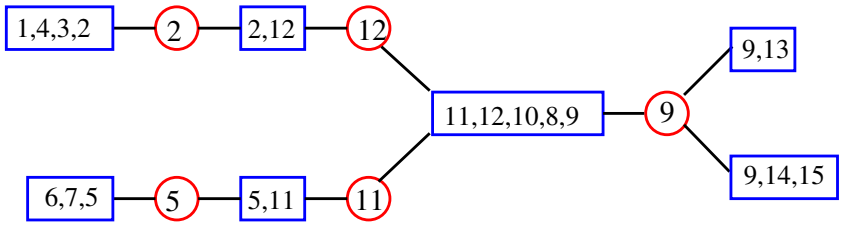


9. ábra. A középső ábra a bal oldali gráf blokkjainak gráfja, a jobboldali pedog a blokk-elvágó gráfja.

A blokk-elvágó gráf pontjai a blokkok, és az elvágópontok. Minden `a`, `b` élére teljesül, hogy `a` elvágópont és `a` eleme a `b` blokknak.



10. ábra. Kétszeresen összefüggő komponensek



11. ábra. Blokk-elvágó gráf

3.3.1. Megvalósítás

```
//Irányítatlan gráf blokk-elvágó fája
#include <bits/stdc++.h>
#define maxN 10001
using namespace std;

int n;//pontok száma
vector<int> G[maxN];
enum Paletta {Feher, Szurke, Fekete};
Paletta Szin[maxN];
int D[maxN];
int L[maxN];
int Bid[maxN];
bool ElvagoP[maxN];
stack<int> V;
vector<vector<int>> Blokk;
vector<int> BFa[maxN];//a blokk-elvágó fa
int ido;
```

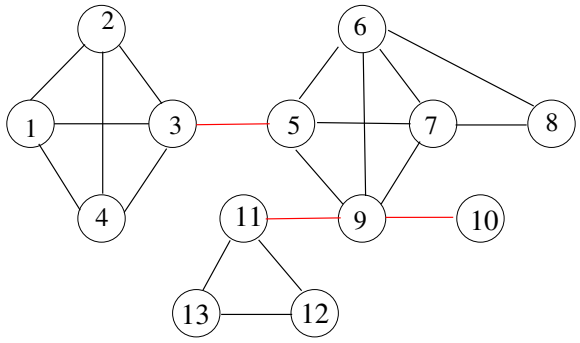
```
void MelyBejar(int p, int os){
  //Global: G, ido, D,Szin,L, V,Blokk
  D[p]=++ido;
  L[p]=ido;
  Szin[p]=Szurke;
  V.push(p);
  for (int q:G[p]){
    if(q==os) continue;
    if (Szin[q]==Feher){//p->q faél;
      MelyBejar(q, p);
      L[p]=min(L[p],L[q]);
      if (L[q]>=D[p]){//találtunk egy komponenst
        ElvagoP[p] = (D[p] > 1 || D[q] > 2);
        Blokk.push_back({p});
        while (Blokk.back().back() != q) {
          Blokk.back().push_back(V.top());
          V.pop();
        }
      }
    }else if (q!=os && Szin[q]==Szurke){//p->q visszaél
      L[p]=min(L[p],D[q]);
    }
  }
  Szin[p]=Fekete;
}
void Komponensek(){
  //global: G, Blokk
  for (int p=1; p<=n; p++) {Szin[p]=Feher; ElvagoP[p]=false;}
  ido=0;
  for (int p=1; p<=n; p++){
    if (Szin[p]==Feher) MelyBejar(p,-1);
  }
}
```

```
151 void BeOlvas();
152 int main (){
153   ios_base::sync_with_stdio(false);
154   cin.tie(NULL);
155   BeOlvas();
156   Komponensek();
157   //a p elvágópont sorszáma Bid[p] lesz
158   int bid=Blokk.size();
159   for (int x = 1; x <= n; x++)
160   if (ElvagoP[x])
161   Bid[x] = bid++;
162   //blokkbeli pontok blokkhoz rendelése a Bid[] függvénnyel
163   //a gráf pontjai a blokkok és elvágópontok azonosítói
164   for(int i=0;i<Blokk.size();i++)
165   for(int x:Blokk[i]){
166     if(ElvagoP[x]){
167       BFa[i].push_back(Bid[x]);
168       BFa[Bid[x]].push_back(i);
169     }else
170     Bid[x]=i;
171   }
172   //blokkok kiíratása
173   cout<<"Blokkok:\n";
174   for(int b=0; b<Blokk.size();b++){
175     cout<<b<<":";
176     for(int x:Blokk[b]) cout<<x<<","; cout<<endl;
177   }cout<<endl;
178   cout<<"elvágópontok és fa-beli azonosítójuk:\n";
179   for(int p=1;p<=n;p++)
180   if(ElvagoP[p]) cout<<Bid[p]<<":"<<p<<endl;
181   //a blokk-elvágó fa kiíratása
182   cout<<"a blokk-elvágó fa:\n";
183   for(int x=0;x<bid;x++){
184     cout<<x<<"->";
185     for(int y:BFa[x]) cout<<y<<","; cout<<endl;
186   }
187 }
```

4. Él-összefüggőség

4.1. definíció. *Azt mondjuk, hogy a* `G = (V, E)` *gráf 2-él-összefüggő, ha összefüggő és bármely élét törölve a gráf továbbra is összefüggő marad. Másképpen fogalmazva, a gráfban nincs híd.*

4.2. definíció. *2-él-összefüggő komponens olyan maximális részgráf, amely 2-él-összefüggő.*



12. ábra. Az 2-él-összefüggő komponensek: `{1, 2, 3, 4}, {5, 6, 7, 8, 9}, {11, 12, 13}, {10}`

Az világos, hogy a komponenseknek nincs közös éle, sem közös pontja.
Tehát a komponenseket megkapjuk, ha töröljük a hidakat. Ez megvalósítható hasonló módon, mint ahogy a kétszeresen összefüggő komponensek kiszámítását tettük, csak most a hidak detektálásakor kell a veremből kiszedni a komponenst alkotó pontokat.

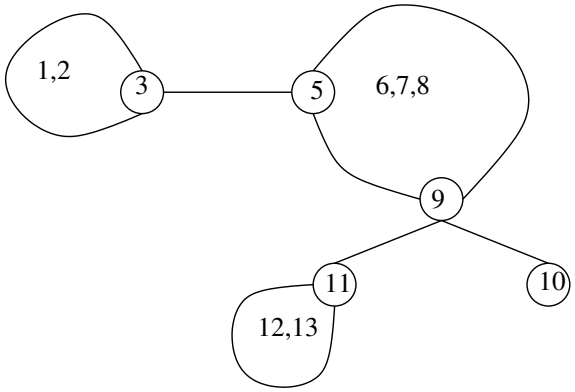
4.1. 2-él-összefüggő komponensek kiszámítása

```
1  //Irányítatlan gráf 2-él-összefüggő komponensek
2  #include <bits/stdc++.h>
3  #define maxN 100001
4  using namespace std;
5
6  int n;//pontok száma
7  vector<int> G[maxN];
8  int D[maxN];
9  int L[maxN];
10 stack<int> V;
11 vector<int> Komp[maxN];
12 int kompid=0;
13 int ido;
```

```
188 void MelyBejar(int p, int os){
189   //Global: G, ido, D,Szin,L, V, Komp, kompid
190   D[p]=++ido;
191   L[p]=ido;
192   V.push(p);
193   for (int q:G[p]){
194     if(q==os) continue;
195     if (D[q]==0){//p->q faél;
196       MelyBejar(q, p);
197       L[p]=min(L[p],L[q]);
198     }else //p->q visszaél
199     L[p]=min(L[p],D[q]);
200   }
201   if (L[p]==D[p]){//találtunk egy komponenst
202     while (V.top() != p) {
203       Komp[kompid].push_back(V.top());
204       V.pop();
205     }
206     int q=V.top(); V.pop();
207     Komp[kompid++].push_back(q);
208   }
209 }
210 void Komponensek(){
211   //Globális: Szin
212   for (int p=1; p<=n; p++) {D[p]=0;}
213   ido=0;
214   for (int p=1; p<=n; p++)
215   if (D[p]==0) MelyBejar(p,-1);
216 }
```

```
void BeOlvas();
int main (){
  ios_base::sync_with_stdio(false);cin.tie(NULL);
  BeOlvas();
  Komponensek();
  cout<<kompid<<endl;
  for(int i=0;i<kompid;i++){
    for(auto p:Komp[i])
    cout<<p<<" "; cout<<endl;
  }
}
void BeOlvas(){
  //Global:n,G
  int m,p,q;
  cin>>n>>m;
  for (int i=0; i<m; i++){
    cin>>p>>q;
    G[p].push_back(q);
    G[q].push_back(p);
  }
}
```

A 2e-összefüggő komponensek nyilvánvalóan diszjunktak.
Konstruálhatunk a 2e-összefüggő komponensekből olyan gráfot, amely körmentes lesz. A `ka` és a `kb` komponens között akkor és csak akkor legyen él, ha van olyan `pa` $\rightarrow$ `pb` él az eredeti gráfban, amely híd, és `pa` $\in$ `ka` és `pb` $\in$ `kb`. Nevezzük ezt a gráfot 2e-komponens-gráfnak.



13. ábra. A példa gráf 2e-komponens-gráfja.

4.2. 2e-komponens-gráf előállítása

Mivel a 2e-komponensek diszjunktak, így az eredeti `G = (V, E)` gráf minden pontjához hozzárendelhetjük a a komponensek előállítása során a komponens `kompid` sorszámát. A 2e-komponens-gráfja pontjai a `kompid` sorszámok lesznek.

```
1  //Irányítatlan gráf 2-él-komponens-gráfja
2  #include <bits/stdc++.h>
3  #define maxN 100001
4  using namespace std;
5
6  int n;//pontok száma
7  vector<int> G[maxN];
8  int D[maxN];
9  int L[maxN];
10 stack<int> V;
11 vector<int> Komp[maxN];
12 int kompid=0;
13 int Hid[maxN];
14 int KompId[maxN];
15 vector<int> e2kFa[maxN];
16 int ido=0;
```

```
238 void MelyBejar(int p, int os){
239   //Global: G, ido, D,Szin,L, V, Komp, kompid
240   D[p]=++ido;
241   L[p]=ido;
242   V.push(p);
243   for (int q:G[p]){
244     if(q==os) continue;
245     if (D[q]==0){//p->q faél;
246       MelyBejar(q, p);
247       L[p]=min(L[p],L[q]);
248       if(L[q]==D[q])
249       Hid[q]=p;
250     }else //p->q visszaél
251     L[p]=min(L[p],D[q]);
252   }
253   if (L[p]==D[p]){//találtunk egy komponenst
254     while (V.top() != p) {
255       int x=V.top(); V.pop();
256       Komp[kompid].push_back(x);
257       KompId[x]=kompid;
258     }
259     int q=V.top(); V.pop();
260     KompId[q]=kompid;
261     Komp[kompid++].push_back(q);
262   }
263 }
264 void Komponensek(){
265 //Globális: Szin
266   for (int p=1; p<=n; p++){
267     D[p]=0; Hid[p]=0; KompId[p]=0;}
268   for (int p=1; p<=n; p++)
269     if (D[p]==0) MelyBejar(p,-1);
270   for(int p=1;p<=n;p++){
271     if(Hid[p]>0){
272       e2kFa[KompId[p]].push_back(KompId[Hid[p]]);
273       e2kFa[KompId[Hid[p]]].push_back(KompId[p]);
274     }
275   }
276 }
```

```
void BeOlvas();
int main (){
  ios_base::sync_with_stdio(false);cin.tie(NULL);
  BeOlvas();
  Komponensek();
  cout<<kompid<<endl;
  for(int i=0;i<kompid;i++){
    cout<<i<<":";
    for(auto p:Komp[i])
    cout<<p<<" "; cout<<endl;
  }cout<<endl;
  for(int x=0;x<kompid;x++){
    cout<<x<<"->";
    for(int y:e2kFa[x])
    cout<<y<<","; cout<<endl;
  }
}
```

5. Feladat 1: Kritikus csomópontok; https:/mester.inf.elte.hu/ Mélységi/91.

Minden számítógépes hálózat csomópontokból és bizonyos csomópontpárok között kiépített közvetlen kétirányú adatátvitelt biztosító kommunikációs vonalakból épül fel. A vizsgált hálózat összefüggő, tehát bármely két csomópont között van kommunikációs útvonal. Lehetnek azonban olyan csomópontok, amelyek meghibásodása esetén a hálózat nem lesz összefüggő. Az ilyen csomópont meghibásodása esetén a hálózat egynél több részhálózattá esik szét, amelyek mind-egyike önmagában összefüggő. A hálózat üzemetetői meg akarják állapítani, hogy melyik az a csomópont, amelynek a meghibásodása esetén a hálózat a legtöbb részhálózatra bomlik szét. Írj programot, amely meghatározza azt a csomópontot, amelynek a meghibásodása esetén a háló-zat a legtöbb részhálózatra bomlik szét!

Bemenet

Bemenet A standard bemenet első sorában két egész szám van, a csomópontok száma ($1 < N \leq 100\,000$), és azon csomópontpárok száma ($1 < M \leq 200\,000$), amelyek között közvetlen adatátvitel van kiépítve. A további M sor mindegyike egy u v számpárt tartalmaz ($1 \leq u \neq v \leq N$), amely azt jelenti, hogy az u és v csomópont között közvetlen kétirányú adatátviteli vonal van kiépítve. Bármely két csomópont között legfeljebb egy közvetlen vonal van. A hálózatra teljesül, hogy bármely két csomópont között létezik útvonal.

Kimenet

Kimenet első sorába annak a csomópontnak a K számot kell írni, amelynek a meghibásodása esetén a há-lózat a legtöbb részhálózatra bomlik szét! A második sorba a szétbomlás utáni részhálózatok számát kell írni. Több megoldás esetén a legkisebb sorszámú csomópontot kell kiírni! Ha nincs kritikus pont a hálózatban, akkor az első és egyetlen sorba a 0 számot kell kiírni!

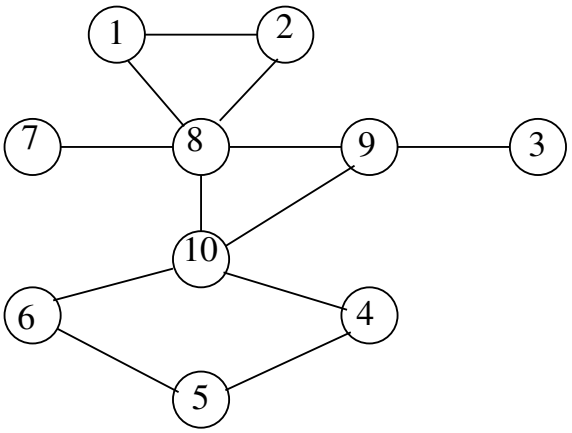
Példa

Bemenet

```
10 12
1 2
2 8
8 1
8 7
8 9
8 10
9 3
9 10
4 5
5 10
5 6
6 10
```

Kimenet

```
8
3
```



14. ábra

Pontozás

Pontozás A pontok `10%`-a kapható olyan tesztekre, ahol $N \leq 16$.
A pontok `20%`-a kapható olyan tesztekre, $N \leq 1000$.
A pontok `70%`-a kapható olyan tesztekre, ahol nincs egyéb korlátozás.

Korlátok

Korlátok $1 < N \leq 100\,000$
$1 \leq M \leq 200\,000$
Időlimit: `0.1` másodperc
Memórialimit: `32` MB.

6. Feladat 2: Hibatűrő hálózat; https:/mester.inf.elte.hu/ Mélységi/89.

Minden számítógépes hálózat csomópontokból és bizonyos csomópontpárok között kiépített közvetlen kétirányú adatátvitelt biztosító kommunikációs vonalakból épül fel. Egy hálózatot hibatűrőnek nevezünk, ha bármely közvetlen vonal meghibásodása esetén is van útvonal bármely két megmaradt csomópont között. Írj programot, amely kiszámítja, hogy legkevesebb hány új közvetlen vonalat kell kiépíteni ahhoz, hogy a hálózat hibatűrő legyen!

Bemenet

Bemenet A standard bemenet első sorában két egész szám van, a csomópontok száma ($1 < N \leq 100\,000$), és azon csomópontpárok száma ($1 < M \leq 200\,000$), amelyek között közvetlen adatátvitel van kiépítve. A további `M` sor mindegyike egy `u v` számpárt tartalmaz ($1 \leq u \neq v \leq N$), amely azt jelenti, hogy az `u` és `v` csomópont között közvetlen kétirányú adatátviteli vonal van kiépítve. Bármely két csomópont között legfeljebb egy közvetlen vonal van. A hálózatra teljesül, hogy bármely két csomópont között létezik útvonal.

Kimenet

Kimenet A standard kimenet első sorába azt a K számot kell írni, amely a legkevesebb olyan új közvetlen vonalak száma, amelyek kiépítésével a hálózat hibatűrővé válik! A további K sor mindegyike egy megépítendő új közvetlen vonal két végpontját tartalmazza! Ha a hálózat már eleve hibatűrő, akkor csak a 0 számot kell kiírni! Több megoldás esetén bármelyik megadható.

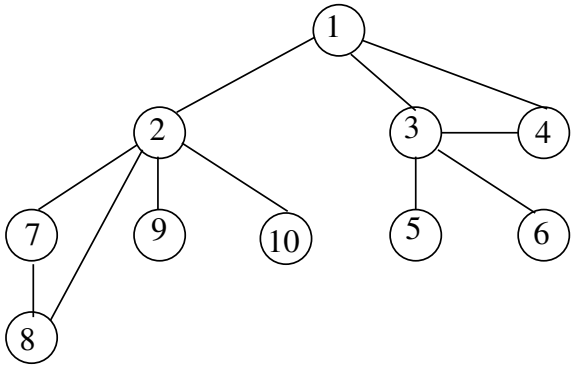
Példa

Bemenet

```
10 11
1 2
1 3
1 4
3 4
3 6
3 5
2 7
2 8
2 9
2 10
7 8
```

Kimenet

```
2
9 6
10 5
```



15. ábra

Pontozás

Pontozás A pontok 50%-a minden tesztesetre, ha a kimenet első sora helyes.
A pontok 10%-a kapható olyan tesztekre, ahol $N < 16$.
A pontok 30%-a kapható olyan tesztekre, ahol a megoldás elemszáma 1.
A pontok 60%-a kapható olyan tesztekre, ahol nincs egyéb korlátozás.

Korlátok

Korlátok $1 < N \leq 100\,000$
$1 \leq M \leq 200\,000$
Időlimit: 0.1 másodperc
Memórialimit: 32 MB.

7. Feladat 3: Turistajárat; https:/mester.inf.elte.hu/ Mélységi/77.

Netura idegenforgalmi ügynökség turista utakat szervez az országban. Minden járat a cég központjából indul és egy adott célállomásra megy meghirdetett útvonalon. Az országban sok helyen van olyan látványosság, ami a Világörökség listáján is szerepel. Minden ilyen látványosság valamely két város közötti útszakasz mentén van. Az ügynökség minden olyan célvárosba indít járatot, amelyhez vezet olyan útvonal, amely áthalad legalább egy látványosság mellett.
Ismerjük az úthálózatot, azaz, hogy mely várospárok között van (kétirányban járható) közvetlen útszakasz. Azt is tudjuk, hogy mely útszakaszok mentén van látványosság. Minden járat a célállomásig különböző városokon át halad.
Adjuk meg azokat a célvárosokat, amelyekbe indit járatot az ügynökség!

Bemenet

Bemenet A standard bemenet első sorában három egész szám van, a városok `N` száma, a közvetlen útszakaszok `M` száma és a látványosságok `K` száma. A városokat az `1, ..., N` számokkal azonosítjuk, az ügynökség az 1. városban van. A további `M` sor mindegyike egy közvetlen útszakaszt ad meg a két végpontjával. Az első `K` közvetlen útszakasz mellett van látványosság. Bármely két város között legfeljebb egy közvetlen útszakasz van.

Kimenet

Kimenet A standard kimenet első sorába azon városok `L` számát kell írni, amelyekhez van olyan útvonal az ügynökség helyétől, amely áthalad legalább egy látványosság mellett! A második sorba kell kiírni ezen városokat, tetszőleges sorrendben.

Példa

Bemenet

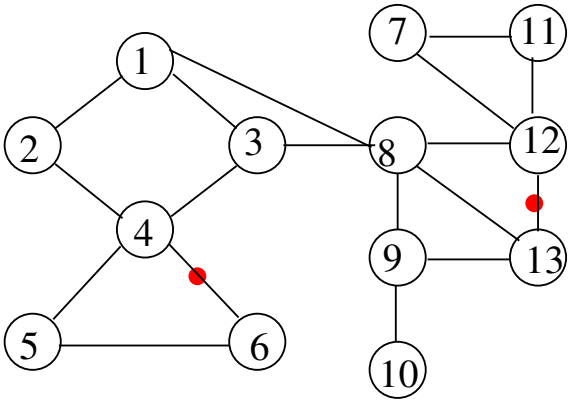
```
13 18 2
4 6
12 13
1 2
1 3
2 4
3 4
4 5
5 6
1 8
3 8
8 9
8 12
8 13
9 10
9 13
7 11
7 12
11 12
```

Kimenet

```
8
5 6 9 10 12 11 13 7
```

Pontozás

10 pont jár olyan tesztekre, ahol $N \leq 100$
10 pont jár olyan tesztekre, ahol bármely két város között pontosan egy útvonal van
10 pont jár olyan tesztekre, ahol csak egy látványosság van



16. ábra

10 pont jár olyan tesztekre, ahol $N \leq 5\,000$
60 pont jár olyan tesztekre, ahol nincs egyéb korlát

Korlátok

$2 \leq N \leq 40\,000$
$1 \leq M \leq 400\,000$
$1 \leq K \leq M$

Időlimit: 0.25 s

Memórialimit: 64 MiB